DISEÑO DE ALGORITMOS

ASESOR:
ING. GERARDO ACEVEDO VEGA

UNIDAD 2 · ACTIVIDAD COMPLEMENTARIA 1

FASES DE LA COMPILACIÓN DE UN PROGRAMA FUENTE

9 DE OCTUBRE DE 2026

GRUPO 8191 · INFORMÁTICA

Alumno: Mauro Noel Sánchez Campos

Universidad Nacional Autónoma de México

Facultad de Contaduría y Administración

Introducción

Un compilador traduce el programa que escribimos en un lenguaje de alto nivel, como C, a código que la máquina puede ejecutar. No lo hace de un solo golpe: el programa pasa por varias fases, y cada una recibe lo que produjo la anterior y entrega algo más cercano al lenguaje de la máquina (Kanagusico et al., 2026). Para explicarlas voy a seguir una sola línea de código a lo largo de todo el proceso:

```
float precio, total;
total = precio * 2 + 5;
```

Las fases de la compilación

Las fases se dividen en dos partes. Las tres primeras forman la etapa de **análisis**, que revisa el programa y entiende qué dice. Las tres últimas forman la etapa de **síntesis**, que construye el programa en lenguaje máquina (Aho et al., 2008).

Análisis léxico

El analizador léxico lee el código fuente carácter por carácter y agrupa los caracteres en **tokens**: palabras reservadas, identificadores, operadores, números y signos de puntuación. También elimina lo que no le sirve al compilador, como espacios y comentarios. Si encuentra un símbolo que no pertenece al lenguaje, marca un error léxico. Con nuestra línea, la salida es:

```
<id, total>  <=>  <id, precio>  <*>  <num, 2>  <+>  <num, 5>  <;>
```

Análisis sintáctico

El analizador sintáctico recibe los tokens y comprueba que estén en un orden que respete la gramática del lenguaje, igual que revisamos que una oración en español tenga sujeto y verbo. Si el orden es correcto, construye un **árbol sintáctico** que muestra la jerarquía de las operaciones. En el ejemplo, el árbol deja claro que primero se multiplica `precio * 2` y después se suma 5, porque la multiplicación tiene mayor precedencia. Si faltara el punto y coma, aquí aparecería un error de sintaxis.

Análisis semántico

Una instrucción puede estar bien escrita y aun así no tener sentido. El análisis semántico revisa el significado: que las variables estén declaradas y que los tipos de

datos sean compatibles. Para eso consulta la tabla de símbolos, donde está guardado que `precio` y `total` son de tipo `float`. Como 2 y 5 son enteros, el compilador agrega una conversión para operar con 2.0 y 5.0. El resultado es un árbol anotado con tipos. Si `precio` no estuviera declarada, el error saldría en esta fase.

Generación de código intermedio

Con el árbol revisado, el compilador produce una representación intermedia: un lenguaje sencillo que no pertenece a ninguna máquina real, pero que ya se parece a las instrucciones de un procesador. Una forma común es el código de tres direcciones, donde cada instrucción tiene como máximo un operador:

```
t1 = precio * 2.0
t2 = t1 + 5.0
total = t2
```

Tener este paso intermedio permite reutilizar la misma parte de análisis para generar programas en distintos procesadores.

Optimización

La optimización mejora el código intermedio para que el programa final sea más rápido o use menos memoria, sin cambiar lo que hace. En el ejemplo, la variable temporal `t2` solo sirve para copiar el resultado, así que se puede eliminar:

```
t1 = precio * 2.0
total = t1 + 5.0
```

En programas grandes esta fase también saca cálculos repetidos fuera de los ciclos o elimina código que nunca se ejecuta.

Generación de código objeto

La última fase traduce el código optimizado a instrucciones de la máquina destino: decide qué registros del procesador usar y en qué orden cargar, operar y guardar los datos. Escrito en un ensamblador simplificado, el ejemplo quedaría así:

```
LDF  R1, precio        ; carga precio en el registro R1
MULF R1, R1, #2.0      ; R1 = R1 * 2.0
ADDF R1, R1, #5.0      ; R1 = R1 + 5.0
STF  total, R1         ; guarda el resultado en total
```

Este código objeto todavía se une con las librerías que necesita el programa (enlazado) para obtener el archivo ejecutable final.

Dos apoyos que trabajan en todas las fases

La **tabla de símbolos** no es una fase, sino una estructura de datos donde el compilador guarda los nombres de variables y funciones, sus tipos y sus parámetros; todas las fases la consultan o la actualizan. El **gestor de errores** recibe los errores que detecta cada fase y los reporta al programador con el número de línea, para que pueda corregirlos (Kanagusico et al., 2026).

Diagrama de las fases

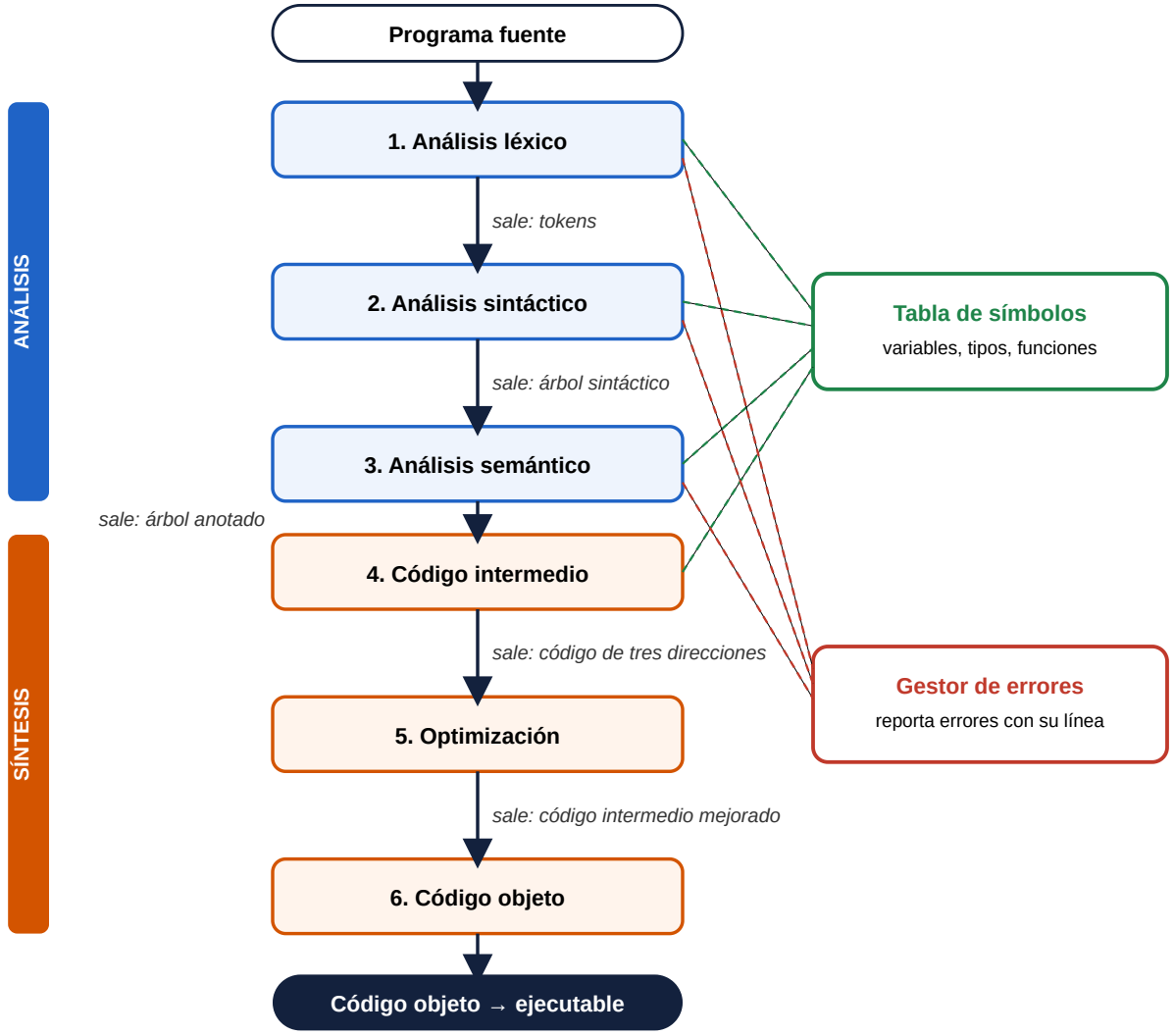


Figura 1. Fases de la compilación y lo que entra y sale de cada una. Las líneas punteadas indican qué fases consultan la tabla de símbolos y reportan errores. Elaboración propia con base en Kanagusico et al. (2026) y Aho et al. (2008).

Conclusiones

Seguir una sola línea de código por todas las fases me ayudó a entender que compilar no es solo "traducir": primero el compilador comprueba que lo escrito sea correcto en forma y en significado, y solo después lo convierte en instrucciones para el procesador. Eso explica por qué los errores de compilación son tan distintos entre sí: un símbolo extraño se detecta en el análisis léxico, un punto y coma faltante en el sintáctico y una variable sin declarar en el semántico.

También me quedó claro por qué existe el código intermedio: separa lo que depende del lenguaje de lo que depende de la máquina, y ahí es donde el compilador puede optimizar. Saber en qué fase aparece cada error me va a servir para corregir mis programas más rápido.

Fuentes de consulta

Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2008). *Compiladores: principios, técnicas y herramientas* (2.ª ed.). Pearson Educación.

Kanagusico Hernández, E. D., Moreno García, R. C. y Hernández Sánchez, E. (2026). *Diseño de algoritmos* [Apunte digital, Licenciatura en Informática]. Universidad Nacional Autónoma de México, Facultad de Contaduría y Administración.